
Mamba for Sound Source Localization: Paper Review and Implementation (Model & Dataset Pipeline)

JungHo Cheon (02521091)

AI-Robotics, KIST School, University of Science & Technology (UST), Seoul, Republic of Korea

Abstract

Sound source localization (SSL) relies on robust spatial feature modeling under noise and reverberation. Recently, Mamba-based state-space models have shown strong sequence modeling capability, motivating their adoption in SSL. In this work, we review TF-Mamba and implement its end-to-end pipeline up to model construction and dataset preparation. TF-Mamba uses the real and imaginary components of STFT coefficients as multi-channel inputs, extracts spectral-spatial representations with a DenseNet-based feature encoder, and performs time-frequency fusion using repeated TF-Mamba blocks. Each block combines temporal and frequency bidirectional Mamba layers, enabling sequence processing along both axes while preserving information via skip connections.

For DOA estimation, the network predicts a 181-point spatial spectrum spanning 0–180°, produced by time pooling and a fully connected layer with tanh activation; training is formulated as MSE minimization between predicted and ground-truth spectra. We additionally implement a simulation dataset pipeline by convolving LibriSpeech utterances with gpuRIR-generated room impulse responses, consistent with the paper’s simulation setup. Future work includes training the implemented model and reporting quantitative localization accuracy.

1. Introduction

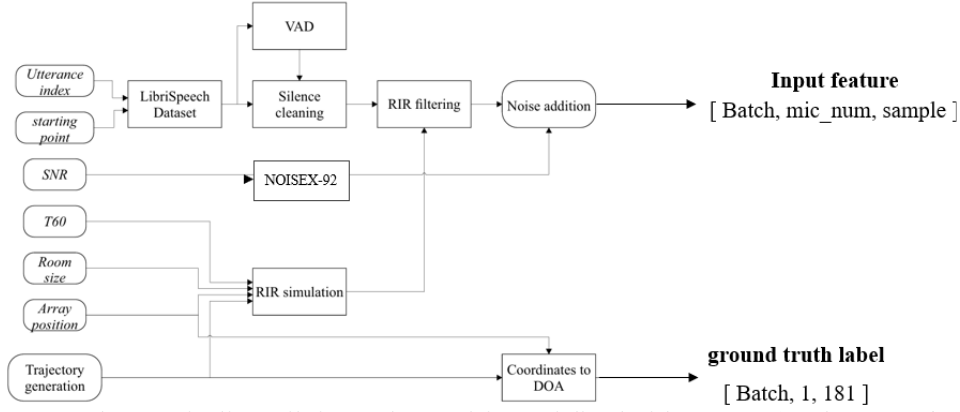
Sound source localization (SSL) estimates the direction of arrival (DoA) from microphone-array signals and provides spatial information that is useful for improving speech separation, recognition, and enhancement. Traditional SSL relies on direct-path spatial cues such as time delay, IPD/ILD, and RTF, as well as SRP-PHAT, but these cues degrade severely under real-world noise, reverberation, overlapping speech, and moving sources. Consequently, deep-learning approaches have become dominant, learning spatial features and longer temporal context with models such as CNNs and RNNs.

TF-Mamba differentiates itself by introducing the SSM-based Mamba to SSL for efficient long-range dependency modeling, applying Mamba along both time and frequency dimensions (a dual-dimension design) and using BiMamba to incorporate bidirectional context. It also adds skip connections to mitigate potential information loss between temporal and frequency Mamba layers.

In implementation, the input consists of the real and imaginary parts of STFT coefficients ($2\times$ the number of microphones), processed by a DenseNet-based feature encoder and TF-Mamba blocks, and the model predicts an 181-point spatial spectrum over 0–180° with an MSE objective against the ground-truth spectrum. In this project, we reviewed the paper, implemented the architecture, and built a simulation data-generation pipeline based on RIR-speech convolution (e.g., LibriSpeech with gpuRIR).

2. Method

2.1 simulation-based dataset

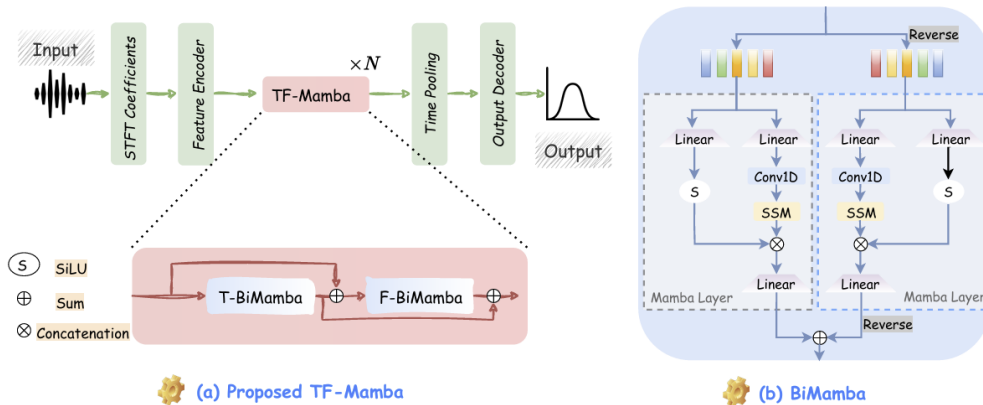


[Fig 1] shows our dataset pipeline: LibriSpeech speech is spatialized with gpuRIR (random room/array/source, Cross3D trajectories) and mixed with NOISEX-92 noise to generate multi-channel inputs and 181-bin DoA labels.

In this work, we constructed a simulation-based SSL (DoA) training dataset following the pipeline shown in Fig. 1. First, we selected utterances from LibriSpeech, cropped audio segments from a specified starting point, and applied VAD to remove silent regions (silence cleaning), so that the input focuses on effective speech content. Next, we used gpuRIR to sample random room sizes, microphone array positions, and source positions, generated the corresponding room impulse responses (RIRs), and convolved the clean speech with the simulated RIRs (RIR filtering) to synthesize multi-channel microphone signals. The reverberation condition was controlled using the T60 parameter.

To model realistic noise conditions, we additionally mixed noise from the NOISEX-92 dataset. Specifically, we prepared several noise types mentioned in the paper as candidates and randomly selected a noise type for each sample. During the noise addition step, we set a target SNR and scaled the noise accordingly so that the desired signal-to-noise ratio is achieved. As a result, the final model input is a multi-channel waveform with the shape [Batch, mic_num, sample], as illustrated in Fig. 1.

The ground-truth labels were generated by converting the simulated source coordinates into DoA. In our implementation, we discretized the 0–180° range at 1° resolution and represented the label as a 181-point spatial spectrum, stored as a tensor of shape [Batch, 1, 181]. Finally, for moving-source scenarios (trajectory generation), we adopted the same source trajectory generation method proposed in Cross3D and incorporated it into the RIR simulation process.[1]



[Fig 2] Overall TF-Mamba pipeline

2.2 TF-Mamba Architecture

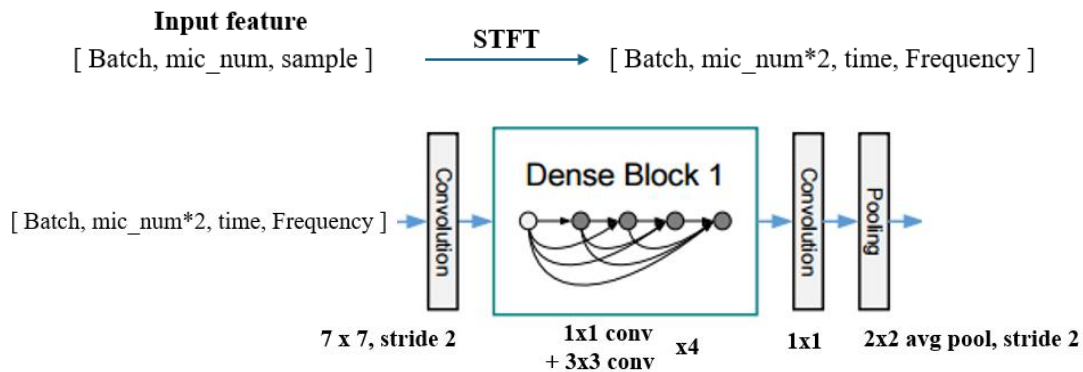
TF-Mamba is a time–frequency network for sound source localization (SSL) whose overall structure is composed of three main modules—(i) a feature encoder, (ii) stacked time–frequency Mamba blocks, and (iii) an output decoder—as illustrated in Fig. 2. The model takes the real and imaginary parts of STFT coefficients as input, resulting in an input channel count that is twice the number of microphones.

2.2.1 Preprocessing & Feature Encoder

Given multi-channel microphone signals, we first compute the short-time Fourier transform (STFT) and construct the network input by splitting complex STFT coefficients into real and imaginary components, which are concatenated along the channel dimension (i.e., $2 \times \text{mic}$ channels).

In our implementation, we follow the paper’s STFT setting (32 ms frame size, 10 ms hop size, and $n_{fft} = 512$; frequency range 100–8000 Hz).[4] This representation preserves phase-related spatial cues while remaining compatible with standard real-valued neural layers.

The resulting time–frequency input is then processed by the feature encoder. TF-Mamba uses a feature encoder consisting of a dilated DenseNet core flanked by two convolutional layers to enhance spectral properties.[2] The encoder outputs a richer feature map that is subsequently fed into the time–frequency Mamba blocks.



[Fig 3] STFT converts the waveform to a $[B, 2 \cdot \text{mic}, T, F]$ feature map (real/imag), which is then encoded by a DenseNet-based feature encoder.

2.2.2 TF-BiMamba (Time–Frequency Fusion)

The core of TF-Mamba is the Time–Frequency Mamba block, which consists of one temporal Mamba layer and one frequency Mamba layer, and additional blocks can be appended by repeating this block N times.

To capture both past and future context (important for non-autoregressive SSL), TF-Mamba adopts bidirectional Mamba (BiMamba), where two SSM branches process the original and reversed sequences in parallel and their outputs are averaged.[3][4]

Inside each TF-Mamba block, the temporal and frequency modules operate on different sequence axes. The temporal Mamba processes each time frame independently with shared parameters, taking a sequence along the frequency axis within a single frame; this allows it to learn inter-frequency dependencies related to spatial/localization cues.

The frequency Mamba then processes each frequency independently (also with shared parameters),

taking a sequence along the time axis for a given frequency; it focuses on estimating direct-path localization features and learning their temporal evolution for moving sources.

Because information extracted in one axis can be weakened after passing through the other axis, TF-Mamba explicitly introduces skip connections to prevent time–frequency information loss.

In addition, the paper notes that skip connections are also used across successive Mamba layers (including within BiMamba).

2.2.3 Decoder (DoA Spectrum Head)

After the stacked TF-Mamba blocks, TF-Mamba applies an average pooling module to reduce the frame rate, followed by a fully connected layer with tanh activation to project features into the target output dimension.

For DOA estimation, the model predicts a spatial spectrum over a half-circle of the linear microphone array, producing an 181-point map that corresponds to 0–180° with 1° resolution. The training objective is defined as MSE between the predicted spatial spectrum and the ground-truth spectrum.

3. Code

This code implements the TF-Mamba model (feature encoder → time/frequency BiMamba blocks → pooling + FC DoA head) and verifies it with a forward-pass run.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from mamba_ssm.modules.mamba_simple import Mamba
import time

class FeatureEncoder(nn.Module):
    def __init__(self, in_channels, growth_rate):
        super().__init__()
        self.conv_1 = nn.Conv2d(in_channels, growth_rate*4, kernel_size=1)
        self.conv_2 = nn.Conv2d(growth_rate*4, growth_rate, kernel_size=3, padding=1)

        self.relu = nn.ReLU(inplace=True)

        self.norm_1 = nn.BatchNorm2d(in_channels)
        self.norm_2 = nn.BatchNorm2d(growth_rate*4)

    def forward(self, x):
        out = self.conv_1(self.relu(self.norm_1(x)))
        out = self.conv_2(self.relu(self.norm_2(out)))

        return torch.cat([x, out], dim=1)
```

```

class TFBimamba(nn.Module):
    """Single-axis BiMamba: residual Mamba along time or frequency."""

    def __init__(self, dim, axis="freq", d_state=16, d_conv=4, expand=2):
        super().__init__()
        assert axis in ("freq", "time")
        self.axis = axis
        self.norm = nn.LayerNorm(dim)
        self.mamba = Mamba(
            d_model=dim,
            d_state=d_state,
            d_conv=d_conv,
            expand=expand,
        )

    def forward(self, x):
        residual = x
        b, c, t, f = x.shape
        if self.axis == "freq":
            seq = x.permute(0, 2, 3, 1).reshape(b * t, f, c) # (B*T, F, C)
        else:
            seq = x.permute(0, 3, 2, 1).reshape(b * f, t, c) # (B*F, T, C)

        seq = self.norm(seq)
        seq = self.mamba(seq)

        if self.axis == "freq":
            seq = seq.reshape(b, t, f, c).permute(0, 3, 1, 2) # (B, C, T, F)
        else:
            seq = seq.reshape(b, f, t, c).permute(0, 3, 2, 1) # (B, C, T, F)

        return seq + residual

```

```

class Bimamba(nn.Module):
    """
    Two BiMamba blocks (frequency then time) connected in a ResNet-style stack.
    """

    def __init__(self, dim, d_state=16, d_conv=4, expand=2):
        super().__init__()
        self.time_block = TFBimamba(dim, axis="time", d_state=d_state, d_conv=d_conv, expand=expand)
        self.freq_block = TFBimamba(dim, axis="freq", d_state=d_state, d_conv=d_conv, expand=expand)

    def forward(self, x):
        t_out = self.time_block(x)
        f_out = self.freq_block(t_out)
        return f_out + t_out

```

```

class TFmamba(nn.Module):
    def __init__(
        self,
        in_channels=4,
        fe_num_layers=2,
        fe_growth_rate=32,
        compression=0.5,
        num_bimamba_blocks=2,
        d_state=16,
        d_conv=4,
        expand=2,
        head_dim=181,
        head_max_angle=180.0,
        aggregate_time_to_one=True,
    ):
        super().__init__()

        self.fe_ch_in = 64

        self.fe_front = nn.Sequential(
            nn.Conv2d(in_channels, self.fe_ch_in, kernel_size=7, stride=2, padding=3),
            nn.BatchNorm2d(self.fe_ch_in),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=1))

        self.fe_layer = []
        for i in range(fe_num_layers):
            self.fe_layer.append(FeatureEncoder(self.fe_ch_in, fe_growth_rate))
            self.fe_ch_in += fe_growth_rate
        self.fe_layer = nn.ModuleList(self.fe_layer)

        self.fe_out_channels = int(self.fe_ch_in * compression)

        self.fe_back = nn.Sequential(
            nn.BatchNorm2d(self.fe_ch_in),
            nn.ReLU(inplace=True),
            nn.Conv2d(self.fe_ch_in, self.fe_out_channels, kernel_size=1),
            nn.AvgPool2d(kernel_size=2, stride=2))

        self.bimamba_blocks = nn.ModuleList(
            [
                Bimamba(self.fe_out_channels, d_state=d_state, d_conv=d_conv, expand=expand)
                for _ in range(num_bimamba_blocks)
            ]
        )

        self.time_pool_kernel = 2
        self.time_pool_stride = 2

        self.head_fc = nn.Linear(self.fe_out_channels, head_dim)
        self.head_max_angle = head_max_angle
        self.aggregate_time_to_one = aggregate_time_to_one

```

```

def forward(self, x):
    x = self.fe_front(x)
    for layer in self.fe_layer:
        x = layer(x)
    x = self.fe_back(x)
    for block in self.bimamba_blocks:
        x = block(x)

    x = F.avg_pool2d(
        x,
        kernel_size=(self.time_pool_kernel, 1),
        stride=(self.time_pool_stride, 1),
    )

    if self.aggregate_time_to_one:
        x = x.mean(dim=2, keepdim=True) # (B, C, 1, F)

    x = x.mean(dim=-1) # (B, C, T)
    x = x.permute(0, 2, 1) # (B, T, C)
    x = torch.tanh(self.head_fc(x)) # (B, T, head_dim) in [0,1]
    x = x * self.head_max_angle # rescale to [0, head_max_angle]

    return x

```

```

if __name__ == "__main__":
    device = "cuda" if torch.cuda.is_available() else "cpu"
    t = time.time()
    x = torch.randn(2, 4, 26, 257, device=device)
    model = Tfmamba(
        in_channels=4,
        fe_num_layers=4,
        fe_growth_rate=32,
        compression=0.5,
        num_bimamba_blocks=5,
    ).to(device)
    y = model(x)
    print("output shape:", y.shape)
    print(time.time() - t)

```

Input feature
[2, 2, 4096]

```

shri_server@shri-MS-7E13:~/Documents/TF_mamba$ py Model.py
output shape: torch.Size([2, 1, 181])

```

[Fig 4] Example forward-pass output shape check using a dummy input (no training), confirming that the TF-Mamba implementation produces the expected DoA spectrum output of [2, 1, 181].

4. Conclusions

I implemented the TF-Mamba model for sound source localization (SSL), targeting robustness to noise and reverberation, and built the simulation-based dataset generation pipeline. However, training and quantitative evaluation were not performed because the ground-truth labeling process has not yet been completed. Moreover, the paper does not fully specify several implementation-critical details (e.g., tensor reshaping/axis handling and internal block configuration), which made reproduction challenging and required additional interpretation. As future work, I will finalize the labeling, train the model, and report quantitative results using the metrics presented in the paper to validate the implementation.

5. references

- [1] D. Diaz-Guerra, A. Miguel, and J. R. Beltran, “Robust Sound Source Tracking Using SRP-PHAT and 3D Convolutional Neural Networks,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 300–311, 2021, doi: 10.1109/TASLP.2020.3040031.
- [2] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely Connected Convolutional Networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2261–2269, doi: 10.1109/CVPR.2017.243.
- [3] X. Zhang, Q. Zhang, H. Liu, T. Xiao, X. Qian, B. Ahmed, E. Ambikairajah, H. Li, and J. Epps, “Mamba in Speech: Towards an Alternative to Self-Attention,” *arXiv preprint arXiv:2405.12609*, 2024, doi: 10.48550/arXiv.2405.12609.
- [4] Y. Xiao and R. K. Das, “TF-Mamba: A Time-Frequency Network for Sound Source Localization,” in *Proc. Interspeech 2025*, Rotterdam, The Netherlands, Aug. 17–21, 2025, pp. 948–952, doi: 10.21437/Interspeech.2025-9.